



ΠΑΝΕΠΙΣΤΗΜΙΟ
ΠΑΤΡΩΝ
UNIVERSITY OF PATRAS

Πολυδιάστατες Δομές Δεδομένων

Project-1 Χειμερινού Εξαμήνου 2024

Διδάσκων: Σ. Σιούτας

Συντάκτες:

Όνομ/νο	ΑΜ	Έτος	Email
Χρήστος Καραμάνος	1072518	6 ^ο	up1072518@ac.upatras.gr
Άγγελος Τσαμόπουλος	1072591	6 ^ο	up1072591@ac.upatras.gr

Επιλέξαμε να υλοποιήσουμε το 1^ο Project από τα δύο της εκφώνησης.

Περιβάλλον Υλοποίησης

Για την εκπόνηση της εργασίας χρησιμοποιήθηκε η γλώσσα προγραμματισμού Python και το περιβάλλον του VSCode. Η δουλειά μοιράστηκε ισόποσα σε όλα τα μέλη.

Dataset

Ως dataset επιλέξαμε το “Used Cars Market Analysis:Scraped Data from Cars24” που βρήκαμε στην ιστοσελίδα Kaggle. Το συγκεκριμένο dataset περιέχει δεδομένα από το Cars24, μια δημοφιλή διαδικτυακή πλατφόρμα για την αγορά και πώληση μεταχειρισμένων αυτοκινήτων. Παρέχει λεπτομερείς πληροφορίες σχετικά με διάφορες καταχωρίσεις μεταχειρισμένων αυτοκινήτων.

Αποτελείται από τις εξής στήλες:

- **Model Name**
- **Price**
- **Manufacturing_year**
- **Engine capacity**
- **Spare key**
- **Transmission**
- **KM driven**
- **Ownership**
- **Fuel type**
- **Imperfections**
- **Repainted Parts**

Περιέχει 1446 εγγραφές που θεωρήσαμε πως είναι αρκετές για το σκοπό μας και το επιλέξαμε, πέρα των άλλων, διότι επιθυμούσαμε τα δεδομένα που θα διαχειριζόμασταν να σχετίζονταν με ένα αντικείμενο που θεωρούμε ενδιαφέρον.

Το link για το dataset:

[“https://www.kaggle.com/datasets/amanrajput16/used-car-price-data-from-cars24”](https://www.kaggle.com/datasets/amanrajput16/used-car-price-data-from-cars24)

Α' Φάση

Αρχικά καλούμαστε να υλοποιήσουμε τις ζητούμενες δομές δεδομένων για διάσταση $k \leq 4$.

Εμείς επιλέξαμε $k = 3$, ώστε το χτίσιμο των δομών να μην είναι ιδιαίτερα απλοϊκό, αλλά και για να αναδείξουμε την διαστατικότητα των συγκεκριμένων δόμων. Τα 3 χαρακτηριστικά του dataset που περιέχει κάθε κόμβος από τις δομές μας είναι: **Price, Engine capacity, KM driven**.

A1) Quad Trees

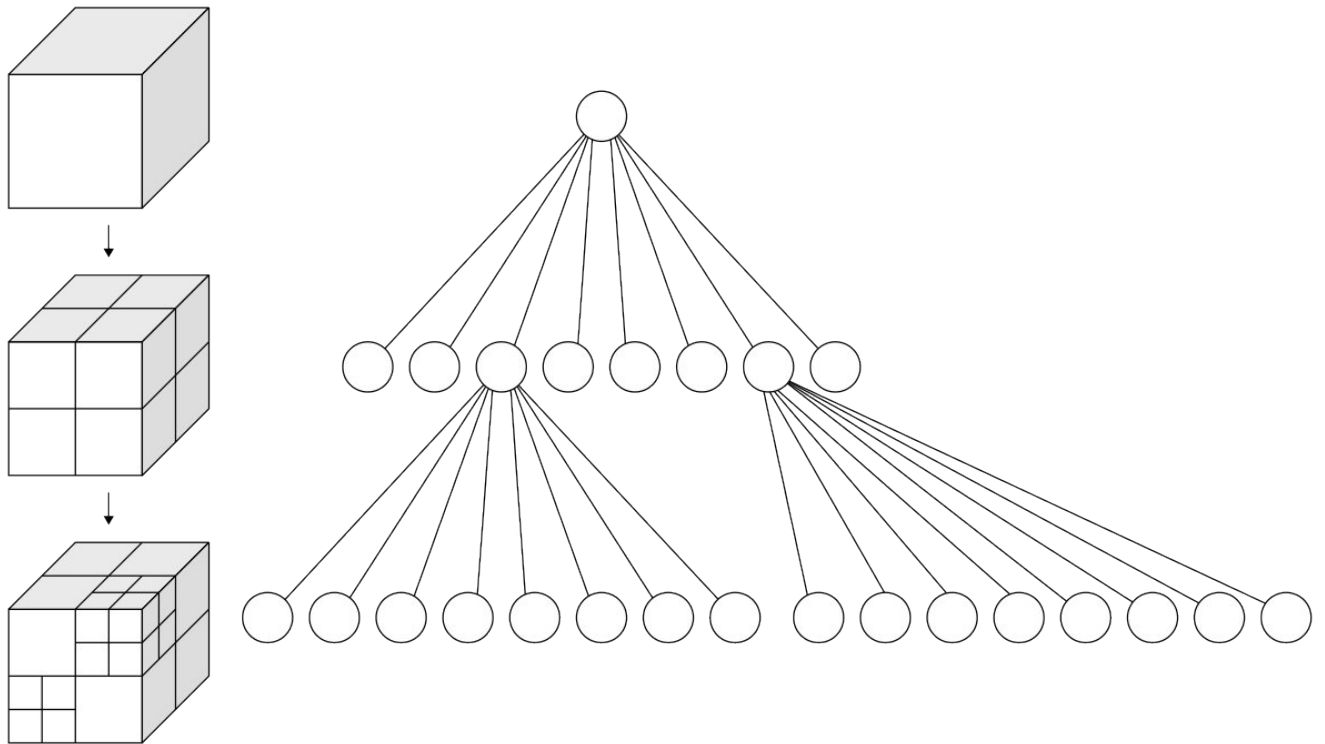
Το Quad Tree είναι δομή δεδομένων που εφαρμόζεται στον δισδιάστατο χώρο. Επειδή εμείς χρειαζόμαστε δομές στον τρισδιάστατο χώρο (αφού έχουμε τρία χαρακτηριστικά ανά αντικείμενο) θα υλοποιήσουμε μία “υποκατηγορία” του Quad Tree, το Octree.

Το Octree είναι μια ιεραρχική δομή δεδομένων που διαμερίζει αναδρομικά έναν τρισδιάστατο χώρο σε όμοιους υποκύβους, επιτρέποντας γρήγορη αποθήκευση και ανάκτηση χωρικών πληροφοριών. Στην κορυφή του – στον ριζικό κόμβο – βρίσκουμε έναν bounding cube, δηλαδή τον ελάχιστο κύβο που περιέχει όλα τα σημεία του dataset. Κάθε εσωτερικός κόμβος μπορεί να υποδιαιρεθεί σε οκτώ ίσα cubes (octants), δημιουργώντας έως και οκτώ δείκτες-παιδιά. Η διαδικασία υποδιαίρεσης συνεχίζεται αναδρομικά, μέχρι να εκπληρωθεί κάποιο κριτήριο τερματισμού, όπως για παράδειγμα η επίτευξη μέγιστου βάθους.

Υπάρχουν δύο κύριες εκδοχές του octree:

- Στο region octree, κάθε κόμβος αντιπροσωπεύει έναν όγκο-voxel στον χώρο, και υποδιαιρείται μόνο εφόσον ο voxel δεν είναι ομοιογενής (π.χ. περιέχει σημεία με διαφορετικές ιδιότητες).
- Στο point octree, κάθε κόμβος αποθηκεύει ένα μοναδικό σημειακό δείγμα (αντικείμενο) (x,y,z) . Κάθε νέο σημείο συγκρίνεται με το κέντρο του τρέχοντος κόμβου και τοποθετείται αναδρομικά στο κατάλληλο octant (με βάση τριψήφιο δυαδικό δείκτη από τις συγκρίσεις σε x , y και z). Η κατανομή αυτή επιτρέπει γρήγορες εισαγωγές points - αντικειμένων στο δέντρο και αποδοτικά range queries σε χωρικά δεδομένα.

Στην παρούσα περίπτωση μας ενδιαφέρει η εκδοχή του point octree και η αναπαράσταση του octree ως δέντρο.



Εικόνα: Παράδειγμα αναπαράστασης Octree.

Αρχικά δημιουργούμε έναν Constructor με όνομα Node ο οποίος χρησιμοποιείται για την δημιουργία ενός κόμβου του δέντρου μας. Κάθε κόμβος έχει attributes (Model Name, Price, Engine capacity και KM driven) και οκτώ δείκτες, έναν για κάθε πιθανό παιδί του.

Στην συνέχεια, με την συνάρτηση InsertCar εισάγεται ένα αυτοκίνητο στην δομή μας ως εξής:

- Γίνεται έλεγχος για κενό δέντρο. Αν δεν υπάρχει ακόμα ριζικός κόμβος (δηλ. node είναι None), δημιουργείται ένας νέος κόμβος με τα δεδομένα του αυτοκινήτου και ορίζεται ως η ρίζα του δέντρου.
- Εξετάζονται τα χαρακτηριστικά Price, Engine capacity και KM driven του κόμβου προς εισαγωγή και συγκρίνονται με εκείνα του κόμβου που εξετάζεται εκείνη τη στιγμή. Μέσω διαδοχικών συγκρίσεων καθορίζεται σε ποιο από τα οκτώ πιθανά παιδιά του κόμβου που εξετάζεται (OOO, OOI, ..., III) πρέπει να μπει το νέο αυτοκίνητο - κόμβος.
- Αν το παιδί που έχει επιλεγεί μέσω των συγκρίσεων είναι κενό (δηλαδή ουσιαστικά δεν υπάρχει), ο νέος κόμβος θα καταχωρηθεί εκεί. Αν όμως

υπάρχει ήδη κόμβος - παιδί, η ίδια διαδικασία επαναλαμβάνεται αναδρομικά (αναδρομική εκτέλεση της παρούσας συνάρτησης), συνεχίζοντας σε χαμηλότερα επίπεδα του δέντρου μέχρι να βρεθεί η κατάλληλη κενή θέση.

- Μόλις το νέο αυτοκίνητο ενταχθεί σε κάποιο φύλλο (ή δημιουργηθεί η ρίζα), η συνάρτηση επιστρέφει αναδρομικά όλους τους κόμβους της διαδρομής, διατηρώντας έτσι τη σωστή δομή του δέντρου.

Με αυτόν τον τρόπο κάθε νέο αυτοκίνητο τοποθετείται στο σημείο του Octree που αντιστοιχεί στη σχετική του θέση στον τρισδιάστατο χώρο που ορίζεται από τα χαρακτηριστικά Price, Engine capacity και KM driven.

Με την συνάρτηση RangeSearch πραγματοποιείται αναζήτηση όλων των κόμβων (αυτοκινήτων) που εμπίπτουν σε ένα τρισδιάστατο διάστημα τιμών που έχει ορίσει ο χρήστης (user query), με τον εξής τρόπο:

- Ο τρέχων κόμβος ελέγχεται πρώτα αν είναι None, και αν είναι τότε διακόπτεται η αναζήτηση και δεν επιστρέφεται κανένα αποτέλεσμα στο υποδέντρο που εξετάζεται εκείνη την στιγμή (με την εκάστοτε εκτέλεση της συνάρτησης).
- Ελέγχεται το αν ο κόμβος που εξετάζεται είναι εντός των ορίων που έχει θέσει ο χρήστης. Συγκρίνονται οι τρεις συντεταγμένες του κόμβου με τα εν λόγω όρια, και αν όλες οι τιμές εμπίπτουν στο δοσμένο εύρος, ο κόμβος προστίθεται στην λίστα CarsInRange.
- Η αναζήτηση περιορίζεται στα υποδέντρα που έχουν νόημα να εξεταστούν. Με άλλα λόγια, γίνεται “κλάδεμα” (pruning) στα υποδέντρα για τα οποία προκύπτει ότι σίγουρα δεν έχουν κόμβους που ανήκουν στο δοσμένο από τον χρήστη διάστημα τιμών.

Πρακτικά, ελέγχονται τα όρια του ερωτήματος σε σχέση με τα αντίστοιχα χαρακτηριστικά του κόμβου για να αποφασιστεί αν έχει νόημα να εξεταστεί το κάθε υποδέντρο.

Για κάθε ένα από τα οκτώ πιθανά παιδιά του κόμβου (OOO, OOI, ..., III), εξετάζεται μία τριάδα συνθηκών ($<$ ή $>=$) ώστε να καθοριστεί αν το υποδέντρο έχει νόημα να υποστεί περαιτέρω εξέταση / αναζήτηση ή όχι.

Μόνο τα υποδέντρα που είναι πιθανό να περιέχουν κόμβους - αντικείμενα εντός των ορίων του ερωτήματος εξετάζονται αναδρομικά (εκτελείται αναδρομικά η συνάρτηση RangeSearch με όρισμα την ρίζα κάθε τέτοιου υποδέντρου).

Στο τέλος της εκτέλεσης, η λίστα CarsInRange περιέχει όλα τα αυτοκίνητα, τα χαρακτηριστικά των οποίων ικανοποιούν ταυτόχρονα όλα τα όρια που έχει θέσει ο χρήστης.

Κατά την διάρκεια της εκτέλεσης του κώδικα, υπολογίζονται οι χρόνοι για την δημιουργία της δομής και για την εκτέλεση του range search, με σκοπό να χρησιμοποιηθούν αργότερα για ανάλυση και σύγκριση.

Παρακάτω παραθέτουμε και ένα παράδειγμα εκτέλεσης του κώδικα:

```
Total construction time: 0.008007049560546875 seconds

-----
User Query
Give the minimum price: 400000
Give the maximum price: 500000
Give the minimum engine capacity: 900
Give the maximum engine capacity: 1000
Give the minimum kilometers driven: 10000
Give the maximum kilometers driven: 20000
-----

Range search finished. Results:

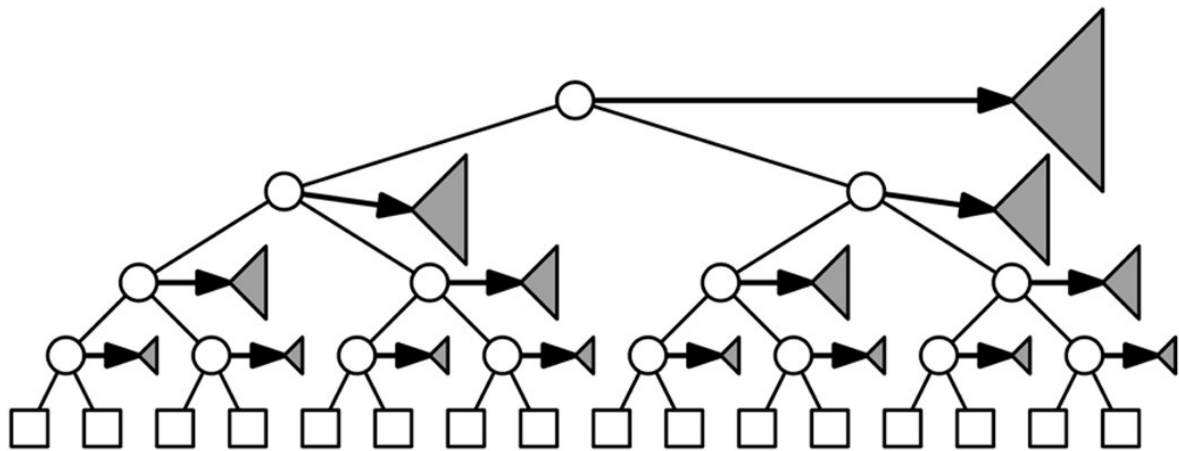
['2018 Maruti Celerio X ZXI AMT', 494000, 998, 14268]
['2021 Maruti S PRESSO VXI+', 450000, 998, 10266]
['2016 Maruti Celerio VXI CNG', 426000, 998, 10347]
['2022 Maruti Alto K10 LXI', 423000, 998, 10776]
['2021 Maruti S PRESSO VXI+', 432000, 998, 12058]
['2023 Maruti Alto K10 VXI PLUS', 489000, 998, 10106]
['2022 Maruti S PRESSO VXI+', 476000, 998, 10815]
['2021 Maruti S PRESSO VXI+', 472000, 998, 10432]
['2020 Maruti S PRESSO VXI (O) AMT', 454000, 998, 13440]
['2022 Maruti S PRESSO VXI+', 455000, 998, 12690]
['2022 Maruti S PRESSO VXI+', 483000, 998, 13814]
['2023 Maruti Alto K10 LXI', 472000, 998, 16628]
```

A2) Range Trees

Τα Range Trees στηρίζονται στην αρχή των Φυλλοπροσανατολισμένων Ισοζυγισμένων Δυαδικών Δέντρων Αναζήτησης (BBST). Όταν πρόκειται για τρισδιάστατα δεδομένα, το 3D Range Tree αποτελείται από μια ιεραρχία πολλών BBST που συνεργάζονται για να δομήσουν τα δεδομένα στις τρεις διαστάσεις. Συγκεκριμένα, αν θεωρήσουμε ότι οι συντεταγμένες των σημείων είναι x , y και z , τότε το δέντρο ξεκινά με ένα BBST που ταξινομεί τις τιμές της διάστασης x . Για κάθε κόμβο αυτού του δέντρου, που

αντιστοιχεί σε μια τιμή x_i , κατασκευάζεται ένα δευτερεύον BBST το οποίο οργανώνει τις τιμές της διάστασης y για όλα τα σημεία με $x=x_i$, καθώς και για όλα τα σημεία που βρίσκονται στα φύλλα των υποδέντρων με ρίζα το x_i . Στη συνέχεια, κάθε κόμβος στο y -BBST συνδέεται με ένα ακόμη BBST που οργανώνει τις τιμές της διάστασης z με τον ίδιο τρόπο.

Στην παρακάτω εικόνα φαίνεται πως το Range Tree διαμορφώνεται όταν οι διαστάσεις είναι 2.



Εικόνα: Παράδειγμα αναπαράστασης 2D Range Tree (εικόνα από διαφάνεια μαθήματος)

Όσον αφορά τον κώδικα, αρχικά διαβάζουμε τα δεδομένα μας από το csv αρχείο και αποθηκεύουμε τις ζητούμενες στήλες σε μία λίστα αποτελούμενη από python tuples που με τη σειρά τους αποτελούνται από 3 τιμές: Price, Engine capacity, KM driven της κάθε εγγραφής.

Για το χτίσιμο του δέντρου δημιουργήσαμε 3 κλάσεις: RangeTree3D, RangeTree2D, RangeTree1D. Ξεκινώντας από τη **RangeTree3D** ο constructor μας δέχεται ως είσοδο τη λίστα με τα δεδομένα και αυτά ταξινομούνται κατά αύξουσα τιμή του x (price), ώστε να μπορέσουμε

εύκολα να βρούμε το median και να φτιάξουμε ισορροπημένο δέντρο. Στη συνέχεια, καλείται η “_build_recursive” για να ξεκινήσει η αναδρομική κατασκευή του κάθε κόμβου του δέντρου. Ο κάθε κόμβος αποτελείται από τα εξής πεδία:

- **value**: το σημείο (x,y,z)
- **range**: ελάχιστη και μέγιστη τιμή του x για το υποδέντρο με ρίζα τον τρέχοντα κόμβο (θα εξηγηθεί αναλυτικά μετέπειτα)
- **left** και **right**: το αριστερό και δεξί υποδέντρο του κόμβου ($x < \text{median}$, $x > \text{median}$ αντίστοιχα) που κατασκευάζεται με αναδρομική κλήση της τρέχουσας κλάσης, αν αυτός δεν είναι φύλλο
- **yz_tree**: εμφωλευμένο yz-tree που προκύπτει από την κλήση του constructor της RangeTree2D κλάσης και δέχεται ως είσοδο τα σημεία του υποδέντρου με ρίζα τον τρέχοντα κόμβο

Για κάθε κόμβο λοιπόν του RangeTree3D κατασκευάζεται ένα υποδέντρο μέσω του **RangeTree2D**. Οι κύριες διαφορές του συγκεκριμένου υποδέντρου είναι ότι η ταξινόμηση των σημείων για την επιλογή του median για κάθε κόμβο γίνεται με βάση την τιμή y (Engine capacity) της τριάδας (x,y,z), το range του κάθε κόμβου καθορίζεται από την ελάχιστη και μέγιστη τιμή του του υποδέντρου του τρέχοντα κόμβου και για την υλοποίηση του εμφωλευμένου z-tree καλείται ο constructor της RangeTree1D.

Η **RangeTree1D** φτιάχνει ένα απλό BBST με βάση τη διάσταση z (KM driven). Να τονίσουμε ότι εδώ ο median περιλαμβάνεται και στο δεξί υποδέντρο, καθώς στην αναζήτηση στο συγκεκριμένο δέντρο κάνουμε απλό φιλτράρισμα σε λίστα, ενώ στα προηγούμενα κάνουμε αναδρομική αναζήτηση με αποτέλεσμα να είναι πιθανό να επισκεφτούμε τον ίδιο κόμβο πάνω από μία φορά.

Σε αυτό το σημείο πρέπει να αναφέρουμε μία σημαντική έννοια των range trees και αυτή είναι τα **canonical subtrees**. Έστω ότι εκτελούμε ένα range search. Canonical subtree είναι ένα υποδέντρο του range tree όλα τα σημεία του οποίου βρίσκονται εξ' ολοκλήρου μέσα στο range της αναζήτησης μας για μία συγκεκριμένη διάσταση. Αυτό σημαίνει ότι για την αναζήτηση μας δε χρειάζεται να εξετάσουμε κάθε κόμβο του υποδέντρου ξεχωριστά, αφού γνωρίζουμε ότι όλοι ανήκουν στο range που ψάχνουμε και άρα τους επιστρέφουμε. Με αυτόν τον τρόπο μειώνεται το πλήθος των κόμβων που επισκεπτόμαστε και βελτιώνουμε το χρόνο αναζήτησης. Αυτό το σκοπό εξυπηρετεί το πεδίο range κάθε κόμβου: κρατάει το εύρος τιμών του υποδέντρου του με βάση την εκάστοτε διάσταση και χρησιμοποιείται στη διαδικασία της αναζήτησης.

Συνεχίζουμε με τις συναρτήσεις `range_query_3D`, `range_query_2D` και `range_query_1D`. Ξεκινώντας με τη **`range_query_3D`**, δέχεται ως είσοδο το κόμβο-ρίζα του x-tree (ο οποίος έχει εμφωλευμένο όλη τη δομή του x-tree) και τα ranges που ψάχνουμε για τις τρεις διαστάσεις (**`x_range`**, **`y_range`**, **`z_range`**). Αρχικά, ανακτά το εύρος τιμών x που καλύπτει το υποδέντρο του κόμβου (**`range`**) και από την εξέταση του προκύπτουν 3 περιπτώσεις:

1. Αν το range του κόμβου είναι υποσύνολο του `x_range` του ερωτήματος, τότε βρήκαμε canonical subtree και προχωράμε στο `range_query_2D` για το yz-tree.
2. Αν δεν υπάρχει καμία επικάλυψη ανάμεσα στο range του κόμβου και στο `x_range`, τότε επέστρεψε κενό
3. Αν υπάρχει μερική επικάλυψη, ελέγχει αν ο ίδιος ο κόμβος ανήκει στο εύρος του ερωτήματος και αν ναι τον προσθέτει στα αποτελέσματα. Έπειτα, συνεχίζει αναδρομικά την αναζήτηση στο αριστερό και δεξί υποδέντρο μέσω της `range_query_3D` και επιστρέφει τα αποτελέσματα.

Αν έχει κληθεί η **range_query_2D** σημαίνει ότι έχουμε βρει canonical subtree στην x διάσταση, οπότε και δέχεται ως είσοδο τον κόμβο-ρίζα του συγκεκριμένου υποδέντρου και τα y_range και z_range του ερωτήματος. Εκτελεί αντίστοιχη διαδικασία με τη range_query_3D, αλλά τώρα εξετάζοντας τις τιμές για τα y και z.

Η **range_query_1D** καλείται μόνο όταν έχει βρεθεί y-canonical subtree. Δέχεται μία λίστα από κόμβους και επιστρέφει μόνο αυτούς των οποίων το z ανήκει στο z_range, αφού έχουν προηγηθεί τα φίλτρα για τις x,y τιμές.

Τέλος, να σημειώσουμε δύο πράγματα:

- Παρά τα ονόματα των κλάσεων, κάθε δέντρο αποθηκεύει κόμβους που αποτελούνται από 3 τιμές (x,y,z). Έχει υλοποιηθεί έτσι η δομή, διότι κατά την διαδικασία “ψαξίματος” ελέγχουμε σε κάθε δέντρο αν ένας κόμβος ανήκει στα αποτελέσματα και αν ναι, τον επιστρέφει. Συνεπώς, πρέπει να κρατάει όλη την τριάδα των τιμών.
- Τα x_range, y_range και z_range δίνονται από το χρήστη μέσω terminal.

Παρακάτω παραθέτουμε και ένα παράδειγμα εκτέλεσης του κώδικα:

```

Building 3D range tree...
Build time for 3D Range Tree: 0.5167 seconds
Enter your 3D range query:
Price range (min max): 577000 718000
Engine capacity range (min max): 796 1197
KM driven range (min max): 47110 60017
Running 3D query...
Found 36 matching cars.
Printing the first 10:
2020 Maruti Baleno ZETA PETROL 1.2 | Price: 631000, Engine: 1197, KM: 55790
2021 Maruti New Wagon-R LXI CNG (O) 1.0 | Price: 585000, Engine: 998, KM: 51151
2020 Maruti Swift VXI | Price: 577000, Engine: 1197, KM: 47110
2021 Maruti Celerio ZXI AMT | Price: 592000, Engine: 998, KM: 48061
2017 Maruti Baleno DELTA CVT PETROL 1.2 | Price: 590000, Engine: 1197, KM: 49352
2021 Maruti Swift VXI | Price: 589000, Engine: 1197, KM: 58463
2020 Maruti Swift VXI | Price: 593000, Engine: 1197, KM: 49583
2018 Maruti Dzire ZXI AMT | Price: 596000, Engine: 1197, KM: 50947
2020 Maruti New Wagon-R ZXI 1.2 AMT | Price: 598000, Engine: 1197, KM: 52423
2020 Maruti Baleno DELTA PETROL 1.2 | Price: 593000, Engine: 1197, KM: 54733

```

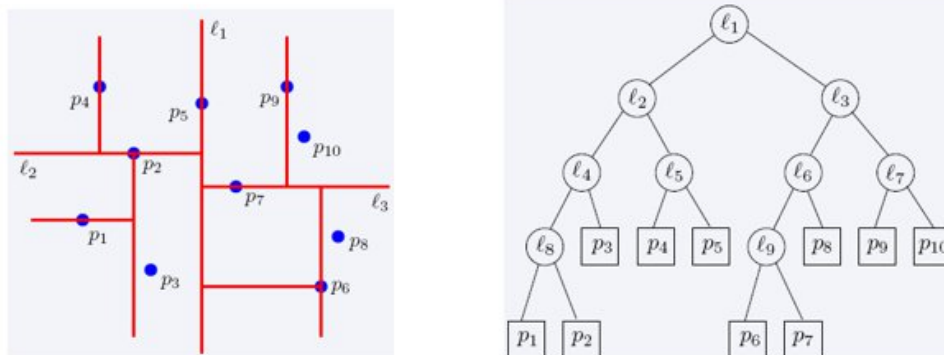
A3) KD-Trees

Τα k-d trees (k-dimensional trees) είναι δομές δεδομένων που χρησιμοποιούνται για την οργάνωση και την αποδοτική αναζήτηση σημείων σε έναν k-διάστατο Ευκλείδειο χώρο. Αποτελούν γενίκευση των δυαδικών δέντρων αναζήτησης (BST) για περισσότερες από μία διαστάσεις και είναι ιδιαίτερα χρήσιμα σε εφαρμογές όπου τα δεδομένα περιέχουν χωρική ή πολυδιάστατη πληροφορία. Οι πιο συνηθισμένες χρήσεις τους είναι για αναζήτηση πλησιέστερων γειτόνων (k-NN), ερωτήματα περιοχής (range queries), ταξινόμηση, σύσταση προϊόντων, πλοήγηση ρομπότ και υπολογιστική γεωμετρία.

Στην περίπτωση μας το δέντρο χτίζεται πάνω σε τρεις διαστάσεις : Price, Engine Capacity, KM Driven. Η κατασκευή του kd-tree γίνεται αναδρομικά. Κάθε κόμβος στο δέντρο χρησιμοποιεί έναν άξονα (axis) για να διαχωρίσει τα δεδομένα, επιλέγοντας κυκλικά έναν από τους k διαθέσιμους. Τώρα που έχουμε 3 χαρακτηριστικά η σειρά είναι Price

- Engine – KM - επανάληψη. Το split γίνεται στον median της επιλεγμένης διάστασης, ώστε να υπάρχει σχετική ισορροπία μεταξύ του αριστερού και δεξιού υποδέντρου. Πιο αναλυτικά για το πώς πραγματοποιείται ο διαχωρισμός, σε έναν 2D χώρο με σημεία (x, y) , το δέντρο εναλλάσσει τη διάσταση μεταξύ x και y , δημιουργώντας κατακόρυφες και οριζόντιες διαχωριστικές γραμμές. Σε 3D χώρο (όπως στην περίπτωση μας), οι διαστάσεις μπορεί να είναι x (Price), y (Engine Capacity) και z (KM Driven), και τα splits γίνονται πάνω σε υπερεπίπεδα που είναι κάθετα σε έναν από αυτούς τους άξονες.

Στην παρακάτω εικόνα φαίνεται ο διαχωρισμός και η διαμόρφωση ενός kd-Tree σε 2 διαστάσεις.



Εικόνα: Παράδειγμα αναπαράστασης 2D kd-Tree (εικόνα από διαφάνεια μαθήματος)

Για την υλοποίηση του κώδικα ακολουθήσαμε την παραπάνω θεωρία που βασίζεται σε αναδρομή και σε διαχωρισμούς του πολυδιάστατου χώρου ανά άξονα. Το δέντρο κατασκευάζεται μέσω της κλάσης [KDTreeCustom](#) όπου αποτελεί τον πυρήνα της υλοποίησης. Στον

constructor της (`_init_`) περιλαμβάνεται όλη η λογική για την αναδρομική κατασκευή του δέντρου. Αρχικά, ελέγχεται αν υπάρχουν διαθέσιμα σημεία. Έπειτα καθορίζεται η διάσταση `axis` με βάση το βάθος (`axis = depth % k`), ταξινομούνται τα σημεία ως προς αυτήν τη διάσταση και επιλέγεται το σημείο στη θέση του median ως κόμβος. Η διαδικασία επαναλαμβάνεται για το αριστερό και δεξί υποσύνολο των δεδομένων, φτιάχνοντας το αριστερό και δεξί υποδέντρο. Η κυκλική εναλλαγή των αξόνων και η επιλογή του median επιτρέπουν ισορροπημένο και αποδοτικό δέντρο, σύμφωνα με τη θεωρία των k-d trees. Η κλάση `KDNode` είναι η βασική μονάδα του δέντρου και κάθε instance της περιέχει:

- Το σημείο `point`
- Τον άξονα διαχωρισμού `axis` (x,y,z)
- Τα υποδέντρα `left` και `right`.

Ο κόμβος λειτουργεί σαν αποθηκευτής δεδομένων αλλά και ως φορέας γεωμετρικής πληροφορίας για το που βρίσκεται στον χώρο.

Για την επιλογή του άξονα ευθύνεται η γραμμή (`axis = depth % k`) ορίζει ποια διάσταση χρησιμοποιείται για διαχωρισμό σε κάθε επίπεδο του δέντρου. Έτσι διασφαλίζεται η κυκλική αλλαγή των αξόνων. Στην συνέχεια το `data.sort(key=lambda x: x[axis])` ταξινομεί τα σημεία σύμφωνα με τον επιλεγμένο άξονα, και το `median = len(data) // 2` επιλέγει το “κέντρο” ως κόμβο. Έτσι, το δέντρο παραμένει **ισοζυγισμένο**, γεγονός που μειώνει σημαντικά τη μέση περίπτωση χρόνου αναζήτησης.

Για την αναπαράσταση του δέντρου χρησιμοποιούμε την μέθοδο `print_tree()` που είναι μια αναδρομική συνάρτηση, ξεκινά από τη ρίζα του δέντρου (`node=None` σημαίνει ξεκινά από `self.node`). Για κάθε κόμβο εκτυπώνει:

- Την κατεύθυνση του (ROOT / LEFT / RIGHT),
- Το βάθος του κόμβου στο δέντρο,
- Τον άξονα στον οποίο έγινε το split (`axis`),

- Το σημείο (point).

Σύμφωνα με την θεωρία πρέπει να εμφανίζονται κυκλικά οι άξονες, στην περίπτωση μας δεν φαίνεται πάντα η κυκλικότητα των αξόνων. Παρότι η εναλλαγή των αξόνων γίνεται κυκλικά βάσει του βάθους (με `axis = depth % k`), αυτό δεν είναι πάντα εμφανές στην εκτύπωση της δομής μέσω της `print_tree()`. Ο λόγος είναι ότι κάθε υποδέντρο μπορεί να έχει διαφορετικό βάθος, ανάλογα με τον αριθμό και την κατανομή των δεδομένων που περιέχει. Αν κάποιο υποδέντρο περιέχει λίγα σημεία, τότε σταματά νωρίτερα — επομένως δεν εκτυπώνονται όλα τα βάθη και οι άξονες σε αυτό το κλαδί. Η `print_tree()` αποτυπώνει τη δυναμική, ασύμμετρη δομή που προκύπτει από τα πραγματικά δεδομένα και όχι ένα ιδεατό πλήρες δέντρο, γι' αυτό και ενδέχεται η κυκλική εναλλαγή να μην φαίνεται εξωτερικά παρότι εφαρμόζεται εσωτερικά σε όλους τους κόμβους.

Στην συνέχεια αναλύουμε την μέθοδο `range_query()` η οποία εφαρμόζει πολυδιάστατη αναζήτηση περιοχής. Δέχεται ως είσοδο ένα σύνολο από διαστήματα (`ranges`) ένα για κάθε διάσταση (Price, Engine Capacity, KM Driven), και επιστρέφει όλα τα σημεία που βρίσκονται εντός όλων αυτών των ορίων. Η υλοποίηση αυτή συμβαίνει με την συνθήκη `in_range = all(ranges[i][0] <= point[i] <= ranges[i][1] for i in range(len(ranges)))` η οποία ελέγχει κάθε διάσταση ανεξάρτητα, και μόνο αν το σημείο ικανοποιεί όλα τα κριτήρια, προστίθεται στα αποτελέσματα. Το πιο κρίσιμο στοιχείο της υλοποίησής είναι το “pruning”. Αυτό σημαίνει “κλάδεμα” δηλαδή παραλείπεις σκόπιμα κάποια υποδέντρα κατά την αναζήτηση επειδή γνωρίζεις ότι δεν περιέχουν λύσεις. Η μέθοδος αυτή ελέγχει αν πρέπει να επισκεφτεί το αριστερό ή/και το δεξί υποδέντρο ανάλογα με τον άξονα στον οποίο βρίσκεται ο κόμβος και τη σχέση της τιμής του σημείου με το αντίστοιχο εύρος : `if point[axis] >= ranges[axis][0]:`

```
self.range_query(node.left.node, ranges, results)
```

```
if point[axis] <= ranges[axis][1]:
```

```
self.range_query(node.right.node, ranges, results)
```

Αν η τιμή του κόμβου είναι έξω από τα όρια του άξονα axis, τότε παραλείπεται το αντίστοιχο υποδέντρο.

Η συνάρτηση `knn_search()` υλοποιεί την αναζήτηση των k πλησιέστερων γειτόνων σε ένα δοσμένο σημείο αναφοράς (query point). Αρχικά, υπολογίζεται η ευκλείδεια απόσταση ανάμεσα στο τρέχον σημείο και το query: `dist = euclidean_distance(point, query_point)`. Η πληροφορία αυτή χρησιμοποιείται για να δημιουργηθεί ένα **max-heap** (best), στο οποίο διατηρούνται τα k καλύτερα (πλησιέστερα) σημεία που έχουν βρεθεί ως τώρα. Αν ο σωρός έχει λιγότερα από k στοιχεία, το νέο σημείο εισάγεται αυτόματα. Αν είναι πλήρης, τότε αντικαθίσταται το πιο μακρινό σημείο μόνο αν το καινούριο είναι πιο κοντινό. Μετά γίνεται ο έλεγχος της αναδρομής στο άλλο υποδέντρο δηλαδή το δέντρο που δεν επιλέχθηκε πρώτα λόγω της θέσης του query point σε σχέση με τον διαχωριστικό άξονα: `if len(best) < k or abs(diff) < - best[0][0]:recursive_search(second.node)`. Εδώ συγκρίνεται η κάθετη απόσταση από τον διαχωριστικό άξονα με τη χειρότερη απόσταση στο heap. Αν υπάρχει πιθανότητα να βρεθεί πιο κοντινό σημείο στο άλλο υποδέντρο, τότε γίνεται αναδρομή και εκεί.

Αυτός ο έλεγχος υλοποιεί τη **θεωρία της χωρικής τομής (hypersphere-hyperplane intersection)** και είναι η βασική τεχνική pruning για k-NN σε KD-Tree. Χάρη σε αυτό, η αναζήτηση παραμένει **αποδοτική και στοχευμένη**, αντί να είναι γραμμική.

Παρακάτω παραθέτουμε και ένα παράδειγμα εκτέλεσης του κώδικα:

```
[RIGHT] Depth 8, Axis 2, Point [737000, 1462, 63804]
[LEFT] Depth 9, Axis 0, Point [747000, 1462, 57301]
[LEFT] Depth 10, Axis 1, Point [743000, 1462, 61765]
[RIGHT] Depth 9, Axis 0, Point [740000, 1462, 86706]
[RIGHT] Depth 7, Axis 1, Point [961000, 1462, 57002]
[LEFT] Depth 8, Axis 2, Point [819000, 1462, 62418]
[LEFT] Depth 9, Axis 0, Point [842000, 1462, 57060]
[LEFT] Depth 10, Axis 1, Point [779000, 1462, 60741]
[RIGHT] Depth 9, Axis 0, Point [840000, 1462, 70110]
[LEFT] Depth 10, Axis 1, Point [757000, 1373, 88132]
[RIGHT] Depth 8, Axis 2, Point [1058000, 1462, 66111]
[LEFT] Depth 9, Axis 0, Point [1069000, 1462, 59304]
[LEFT] Depth 10, Axis 1, Point [999000, 1462, 54421]
[RIGHT] Depth 9, Axis 0, Point [989000, 1462, 68282]
```

Build time for 3D KD-Tree: 0.0142 seconds

Enter your 3D range query:

Price range (min max): 916000 1000000

Engine capacity range (min max): 1000 2000

KM driven range (min max): 45000 60000

Query time for KD-Tree: 0.1052 seconds

Αποτελέσματα Range Query:

```
Model: 2019 Maruti XL6 ZETA AT | Price: 921000 | Engine: 1462 | KM: 54342
Model: 2022 Maruti XL6 ZETA MT | Price: 973000 | Engine: 1462 | KM: 49458
Model: 2022 Maruti Ciaz ZETA 1.5 SHVS MT PETROL | Price: 916000 | Engine: 1462 | KM: 46533
Model: 2019 Maruti XL6 ZETA AT | Price: 956000 | Engine: 1462 | KM: 48847
Model: 2020 Maruti Vitara Brezza ZXI PLUS AT SHVS | Price: 981000 | Engine: 1462 | KM: 50421
Model: 2021 Maruti XL6 ALPHA MT | Price: 1016000 | Engine: 1462 | KM: 48740
Model: 2021 Maruti XL6 ZETA MT | Price: 980000 | Engine: 1462 | KM: 53293
Model: 2021 Maruti Ciaz ALPHA AT 1.5 SHVS PETROL | Price: 961000 | Engine: 1462 | KM: 57002
Model: 2022 Maruti XL6 ZETA MT | Price: 1069000 | Engine: 1462 | KM: 59304
Model: 2021 Maruti XL6 ZETA MT | Price: 999000 | Engine: 1462 | KM: 54421
```

Enter your 3D point for k-NN search:

Enter price (x-axis): 763000

Enter engine capacity (y-axis): 1462

Enter KM driven (z-axis): 73050



Τα 3 πλησιέστερα αυτοκίνητα:

```
Model: 2016 Maruti Vitara Brezza ZDI | Price: 751000 | Engine: 1248 | KM: 76173
Model: 2019 Maruti Ciaz ALPHA 1.5 SHVS PETROL | Price: 756000 | Engine: 1462 | KM: 59733
Model: 2016 Maruti Vitara Brezza ZDI PLUS DUAL TONE | Price: 747000 | Engine: 1248 | KM: 72213
```

A4) R Trees

Τα R-trees είναι μια κατηγορία πολυδιάστατων δομών δεδομένων που χρησιμοποιούνται ως μια αποτελεσματική μέθοδος οργάνωσης και ανάκτησης πολυδιάστατων πληροφοριών σε περιπτώσεις που το κάθε αντικείμενο έχει μια **χωρική** οντότητα (είτε σημείο, είτε ορθογώνιο).

Το σημείο κλειδί εδώ είναι ότι σε σχέση με τις προηγούμενες δομές, τα R-trees δεν διασπάνε τον χώρο σε διαμερίσματα αλλά σε ορθογώνιες περιοχές (MBR – minimum bounding rectangles) που περιέχουν αρκετά κοντινά χωρικά αντικείμενα.

Όπως και στις προηγούμενες δομές, έτσι και τώρα δουλεύουμε πάνω σε τρισδιάστατα δεδομένα, δηλαδή η υλοποίησή μας αφορά μια 3D εκδοχή του R-tree. Από το csv αρχείο, για τις 3 διαστάσεις χρησιμοποιούμε πάλι τα χαρακτηριστικά **Price**, **Engine Capacity** και **KM driven**. Δηλαδή, έχουμε τον άξονα x που ανταποκρίνεται στην τιμή price του csv αρχείου, ο y στην engine capacity και ο z στην km driven.

Κάθε κόμβος περιέχει μια συλλογή από ορθογώνιες περιοχές (MBR) και κάθε μια αντιπροσωπεύει μια υποδομή του δένδρου (ή μια συλλογή σημείων αν πρόκειται για φύλλο).

Η παραδοχή που κάναμε είναι ότι κάθε κόμβος μπορεί να έχει μέχρι 4 παιδιά και σε περίπτωση εισαγωγής νέου σημείου που δεν χωράει στο κατάλληλο φύλλο (ή κόμβο), κάνουμε split.

Για την διαδικασία του split, χρησιμοποιήσαμε το **linear split** του οποίου η διαδικασία περιγράφεται παρακάτω:

- **Επιλογή σπόρων (seeds).** Αφορά την επιλογή δυο αρχικών seeds που βρίσκονται όσο πιο απομακρυσμένα μεταξύ τους. Ο αλγόριθμος συγκρίνει τα ζεύγη και επιλέγει αυτό με την μεγαλύτερη κανονικοποιημένη απόσταση.

- **Κατάτμηση καταχωρήσεων σε ομάδες.** Μετά την επιλογή που κάναμε, όλα τα υπόλοιπα στοιχεία τοποθετούνται σε μια από τις 2 ομάδες, ανάλογα με το πόσο αυξάνεται το εμβαδόν του ορθογωνίου. Αφού υπολογιστεί το νέο μέγεθος του enlargement του κάθε MBR αν προστεθεί στην ομάδα, εν τέλει το στοιχείο τοποθετείται στην ομάδα με το λιγότερο enlargement.
- **Ενημέρωση των MBRs.** Κάθε μια από τις 2 νέες ομάδες έχει το δικό της MBR. Μετά το μοίρασμα, το R-tree ενημερώνεται ώστε κάθε ομάδα να περιέχει όλα τα χαρακτηριστικά των περιεχομένων της.
- **Ενημέρωση του parent.** Μετά το split ο κόμβος που έγινε split ήταν το root, το R-tree δημιουργεί μια νέα ρίζα. Η νέα ρίζα περιέχει 2 καταχωρήσεις, μια ανά μια καινούργια ομάδα. Η κάθε μια καταχώρηση περιέχει το MBR και μια αναφορά προς το νέο παιδί. Εναλλακτικά, αν το split αφορά εσωτερικό κόμβο, το parent ενημερώνεται αφαιρώντας το παλιό node και προσθέτοντας 2 νέα.
- Αν ο parent προσθέτοντας 2 νέες καταχωρήσεις ξεπεράσει τα 4 παιδιά γίνεται και αυτός split με την σειρά του.

Όσον αφορά τον κώδικα, η όλη διαδικασία ξεκινάει με την μέθοδο **RTree** όπου παίρνει σαν είσοδο το μέγιστο πλήθος παιδιών (αντικειμένων) που έχει κάθε κόμβος (δηλαδή 4). Η μέθοδος αυτή αρχικοποιεί το R-tree δημιουργώντας ένα αντικείμενο της κλάσης R-tree. Ο constructor θέτει το μέγιστο πλήθος εισόδων στην μεταβλητή του αντικειμένου και έπειτα δημιουργεί τον κόμβο ρίζα καλώντας τη **RTreeNode**. Στην αρχή, το δέντρο είναι μόνο ένας κόμβος που κρατάει αντικείμενα (points) μέχρι να ξεπεράσει τα τέσσερα όπου θα κάνει split.

Έπειτα ορίζεται η μέθοδος **insert** της κλάσης **RTree** όπου δείχνει πως εισάγουμε αντικείμενα (ή παιδιά). Καλείται από την

main όπου έχουμε πάρει κάθε εγγραφή από το csv αρχείο και την έχουμε βάλει σε ένα λεξικό με το όνομα `point`. Περιέχει την μέθοδο **`create_mbr_from_point`** όπου παίρνει κάθε τρισδιάστατο σημείο που υπάρχει και φτιάχνει ένα MBR για κάθε σημείο όπου στην ουσία είναι το ίδιο το σημείο. Μετά ακολουθεί η μέθοδος **`choose_leaf`** όπου διαλέγουμε σε ποιο φύλλο θα γίνει εισαγωγή του στοιχείου (ή MBR). Το κριτήριο είναι να διαλέξει εκείνο το φύλλο που θα προκαλέσει το λιγότερο enlargement. Αφού γίνει αυτό, καλείται η **`add_entry`** για να γίνει εισαγωγή του στοιχείου στο επιλεγμένο φύλλο. Ταυτόχρονα, γίνεται ενημέρωση του εμβαδόν του MBR και αν δεν χωράει το στοιχείο στο φύλλο (είναι 4) τότε καλείται η μέθοδος `split`.

Η κλάση **`RTreeNode`** ορίζει και αντιπροσωπεύει κάθε κόμβο (φύλλο) του R-Tree. Στην ουσία ο constructor εδώ ορίζει 4 μεταβλητές που είναι η `entries` (λίστα (MBR,object) για φύλλο ή (MBR,child_node) για εσωτερικό κόμβο), την `is_leaf` που δηλώνει αν ο κόμβος είναι ή όχι φύλλο, την `parent` που είναι ένας δείκτης προς τον γονέα του κόμβου και την `mbr` που είναι το MBR του κόμβου.

Με αυτή την διαδικασία, φτιάχνεται το R-tree έχοντας πάρει τα στοιχεία του csv αρχείου.

Στο τέλος, έχουμε υλοποιήσει την συνάρτηση **`print_tree_stats`** για να επιβεβαιώσουμε ότι ο κώδικας τρέχει σωστά και για να έχουμε μια εικόνα για την δομή του R-tree (τι βάθος έχει, πόσους κόμβους περιέχει κτλ). Επισυνάπτεται screenshot της παραπάνω εκτέλεσης:

```
Inserted 1445 points into the R-tree.  
Tree depth: 8  
Total nodes: 815  
Root has 2 entries
```

Για το range query, υλοποιήθηκε η συνάρτηση **get_range_input** όπου ο χρήστης εισάγει την μικρότερη και την μέγιστη τιμή που αυτός θέλει για την καθεμία διάσταση. Έπειτα, η συνάρτηση **range_query** καλείται κατασκευάζοντας το MBR του ερωτήματος και ψάχνοντας τα απαραίτητα στοιχεία με την βοήθεια της συνάρτησης **intersects** (αν το MBR του ερωτήματος τέμνει το MBR ενός κόμβου ακολουθούμε τα παιδιά αυτού του κόμβου και συνεχίζουμε αναδρομικά, αλλιώς κάνουμε pruning το κλαδί που συνεχίζει από αυτό).

Ακολουθεί screenshot που επεβεβαιώνει την παραπάνω λειτουργία:

```
Define your 3D range query:
Enter Price (X) range: 200000 500000
Enter Engine (Y) range: 1000 1300
Enter KM driven (Z) range: 0 60000

Found 99 matching points:
→ 2013 Maruti Swift Dzire VDI: [487000, 1248, 40081]
→ 2016 Maruti Baleno DELTA PETROL 1.2: [489000, 1197, 29235]
→ 2017 Maruti IGNIS DELTA 1.2 AMT: [471000, 1197, 37621]
→ 2017 Maruti Baleno DELTA PETROL 1.2: [489000, 1197, 46407]
→ 2016 Maruti Baleno DELTA PETROL 1.2: [481000, 1197, 59200]
→ 2017 Maruti IGNIS ZETA 1.2 AMT: [461000, 1197, 30087]
→ 2017 Maruti IGNIS SIGMA 1.2: [385000, 1197, 51758]
→ 2014 Maruti Ritz VXI: [269000, 1197, 57273]
→ 2019 Maruti New Wagon-R VXI (O) 1.2 AMT: [500000, 1197, 16796]
→ 2018 Maruti IGNIS DELTA 1.2: [451000, 1197, 34558]
```

Έχουμε υλοποιήσει και άλλες συναρτήσεις για να επιβεβαιώσουμε την σωστή λειτουργία του προγράμματος αλλά τις έχουμε βάλει σε σχόλια για να μην γίνεται εκτύπωση πολλών πληροφοριών όταν τρέχουμε το πρόγραμμα.

Οι συναρτήσεις είναι οι εξής:

- **print_some_leaf_points:** Διασχίζει το δέντρο μέχρι να βρει 10 σημεία τα οποία είναι αποθηκευμένα στα φύλλα. Έτσι επιβεβαιωνόμαστε ότι οι εγγραφές του csv αποθηκεύονται στα φύλλα.

Screenshot Λειτουργίας

```
Point: {'model': '2011 Maruti Alto K10 VXi', 'price': 139000, 'engine': 998, 'km': 83452}
Point: {'model': '2017 Maruti Wagon R 1.0 VXi AMT', 'price': 421000, 'engine': 998, 'km': 19767}
Point: {'model': '2012 Maruti Wagon R 1.0 LXI', 'price': 256000, 'engine': 998, 'km': 55994}
Point: {'model': '2022 Maruti S PRESSO VXi+', 'price': 482000, 'engine': 998, 'km': 28393}
Point: {'model': '2022 Maruti S PRESSO VXi', 'price': 454000, 'engine': 998, 'km': 17426}
Point: {'model': '2021 Maruti S PRESSO VXi+', 'price': 417000, 'engine': 998, 'km': 63516}
Point: {'model': '2020 Maruti S PRESSO VXi+', 'price': 402000, 'engine': 998, 'km': 26931}
Point: {'model': '2023 Maruti Alto K10 VXi', 'price': 457000, 'engine': 998, 'km': 9048}
Point: {'model': '2014 Maruti Wagon R 1.0 LXI', 'price': 326000, 'engine': 998, 'km': 53063}
Point: {'model': '2014 Maruti Wagon R 1.0 LXI', 'price': 294000, 'engine': 998, 'km': 45538}
```

- **print_tree_structure:** Μας δείχνει την ιεραρχία του R-tree δέντρου, τι εμβαδόν πιάνει το MBR του κάθε κόμβου, σε ποιο επίπεδο βρίσκεται ένας κόμβος, αν είναι φύλλο και πόσα παιδιά έχει (entries).

Screenshot Λειτουργίας

```

Level 6 | Leaf: False | Entries: 4
MBR: [367000, 692000], [1197, 1197], [15178, 105779]
  Level 7 | Leaf: True | Entries: 3
    MBR: [367000, 721000], [1197, 1197], [38807, 73631]
  Level 7 | Leaf: True | Entries: 4
    MBR: [384000, 692000], [1197, 1197], [19437, 65568]
  Level 7 | Leaf: True | Entries: 1
    MBR: [376000, 376000], [1197, 1197], [104807, 104807]
  Level 7 | Leaf: True | Entries: 4
    MBR: [564000, 620000], [1197, 1197], [15178, 105779]
Level 6 | Leaf: False | Entries: 3
MBR: [348000, 662000], [1197, 1197], [32730, 64003]
  Level 7 | Leaf: True | Entries: 4
    MBR: [411000, 619000], [1197, 1197], [32730, 59700]
  Level 7 | Leaf: True | Entries: 4
    MBR: [361000, 662000], [1197, 1197], [39945, 64003]
  Level 7 | Leaf: True | Entries: 4
    MBR: [348000, 495000], [1197, 1197], [41074, 61266]
Level 5 | Leaf: False | Entries: 2
MBR: [333000, 668000], [1196, 1197], [32533, 69389]
  Level 6 | Leaf: False | Entries: 2
    MBR: [333000, 668000], [1196, 1197], [61641, 69389]
  Level 7 | Leaf: True | Entries: 1
    MBR: [333000, 333000], [1197, 1197], [69389, 69389]
  Level 7 | Leaf: True | Entries: 4
    MBR: [417000, 668000], [1196, 1197], [28300, 62412]
  Level 6 | Leaf: False | Entries: 1
    MBR: [420000, 473000], [1196, 1197], [32533, 43731]
  Level 7 | Leaf: True | Entries: 3
    MBR: [381000, 473000], [1196, 1197], [32533, 48661]
Level 3 | Leaf: False | Entries: 1
MBR: [450000, 991000], [1197, 1197], [5154, 89872]
  Level 4 | Leaf: False | Entries: 1
    MBR: [450000, 991000], [1197, 1197], [5154, 89872]

```

- **count_nodes_by_level:** Μετράει πόσοι κόμβοι υπάρχουν ανά επίπεδο. Επιβεβαιώνεται παρακάτω ότι δουλεύει σωστά (level 0 είναι ένας κόμβος και όσο προχωράμε επιπέδα τόσο αυξάνονται οι κόμβοι).

Screenshot Λειτουργίας

```

Node counts by level:
Level 0: 1 node(s)
Level 1: 2 node(s)
Level 2: 5 node(s)
Level 3: 12 node(s)
Level 4: 30 node(s)
Level 5: 65 node(s)
Level 6: 185 node(s)
Level 7: 515 node(s)

```

- **print_mbrs_at_levels:** Εκτυπώνει τα MBRs των level 0 και 1. Παρατηρούμε παρακάτω το MBR του root και τα MBRs των 2 παιδιών του και πόσα είναι τα παιδιά τους αντίστοιχα.

Screenshot Λειτουργίας

```
Level 0 | Leaf: False | Entries: 2
  MBR of this node: X=[139000, 1599000], Y=[796, 1462], Z=[1207, 124716]
  | Child MBR: X=[139000, 745000], Y=[796, 998], Z=[1207, 123621]
  | Child MBR: X=[182000, 1599000], Y=[1196, 1462], Z=[2462, 124716]

Level 1 | Leaf: False | Entries: 2
  MBR of this node: X=[139000, 745000], Y=[796, 998], Z=[1207, 123621]

Level 1 | Leaf: False | Entries: 3
  MBR of this node: X=[182000, 1599000], Y=[1196, 1462], Z=[2462, 124716]
```

- **print_leaf_contents:** Εκτυπώνει τα σημεία που βρίσκονται στο MBR των φύλλων (έχουμε βάλει όριο να εκτυπώνει 5 φύλλα-κόμβους). Παρατηρούμε ότι δεν υπάρχει κόμβος που να έχει πάνω από 4 σημεία.

Screenshot Λειτουργίας

```

Level 7 | Leaf: True | Entries: 3
MBR: X=[139000, 421000], Y=[998, 998], Z=[19767, 83452]
  ↳ Point: {'model': '2011 Maruti Alto K10 VXI', 'price': 139000, 'engine': 998, 'km': 83452}
  ↳ Point: {'model': '2017 Maruti Wagon R 1.0 VXI AMT', 'price': 421000, 'engine': 998, 'km': 19767}
  ↳ Point: {'model': '2012 Maruti Wagon R 1.0 LXI', 'price': 256000, 'engine': 998, 'km': 55994}

Level 7 | Leaf: True | Entries: 4
MBR: X=[402000, 482000], Y=[998, 998], Z=[17426, 63516]
  ↳ Point: {'model': '2022 Maruti S PRESSO VXI+', 'price': 482000, 'engine': 998, 'km': 28393}
  ↳ Point: {'model': '2022 Maruti S PRESSO VXI', 'price': 454000, 'engine': 998, 'km': 17426}
  ↳ Point: {'model': '2021 Maruti S PRESSO VXI+', 'price': 417000, 'engine': 998, 'km': 63516}
  ↳ Point: {'model': '2020 Maruti S PRESSO VXI+', 'price': 402000, 'engine': 998, 'km': 26931}

Level 7 | Leaf: True | Entries: 4
MBR: X=[242000, 457000], Y=[998, 998], Z=[9048, 53063]
  ↳ Point: {'model': '2023 Maruti Alto K10 VXI', 'price': 457000, 'engine': 998, 'km': 9048}
  ↳ Point: {'model': '2014 Maruti Wagon R 1.0 LXI', 'price': 326000, 'engine': 998, 'km': 53063}
  ↳ Point: {'model': '2014 Maruti Wagon R 1.0 LXI', 'price': 294000, 'engine': 998, 'km': 45538}
  ↳ Point: {'model': '2014 Maruti Alto K10 VXI', 'price': 242000, 'engine': 998, 'km': 49148}

Level 7 | Leaf: True | Entries: 1
MBR: X=[327000, 327000], Y=[998, 998], Z=[115256, 115256]
  ↳ Point: {'model': '2014 Maruti Wagon R 1.0 LXI', 'price': 327000, 'engine': 998, 'km': 115256}

Level 7 | Leaf: True | Entries: 4
MBR: X=[226000, 443000], Y=[998, 998], Z=[30552, 73539]
  ↳ Point: {'model': '2019 Maruti New Wagon-R VXI 1.0', 'price': 443000, 'engine': 998, 'km': 53840}
  ↳ Point: {'model': '2014 Maruti Alto K10 VXI', 'price': 226000, 'engine': 998, 'km': 36066}
  ↳ Point: {'model': '2018 Maruti Wagon R 1.0 VXI', 'price': 389000, 'engine': 998, 'km': 30552}
  ↳ Point: {'model': '2016 Maruti Celerio VXI AMT', 'price': 351000, 'engine': 998, 'km': 73539}

```

- **show_level1_children_targets:** Εκτυπώνει το root και κατόπιν όλα τα παιδιά του root (level 1) και τα παιδιά αυτών (level2).

Screenshot Λειτουργίας

```

2018 MARCEL TOMIS DELTA 1.2: [431000, 1197, 34358]
Root Node MBR:
X=[139000, 1599000], Y=[796, 1462], Z=[1207, 124716]
Root has 2 children:

◆ Level 1 Node 1
MBR: X=[139000, 745000], Y=[796, 998], Z=[1207, 123621]
Is Leaf: False | Entries: 2
└─ Points to:
  • Entry 1:
    MBR: X=[139000, 745000], Y=[998, 998], Z=[1207, 123621]
    → Node | Leaf: False | Entries: 2
  • Entry 2:
    MBR: X=[182000, 446000], Y=[796, 998], Z=[18317, 110664]
    → Node | Leaf: False | Entries: 3

◆ Level 1 Node 2
MBR: X=[182000, 1599000], Y=[1196, 1462], Z=[2462, 124716]
Is Leaf: False | Entries: 3
└─ Points to:
  • Entry 1:
    MBR: X=[462000, 1155000], Y=[1197, 1373], Z=[20619, 111621]
    → Node | Leaf: False | Entries: 2
  • Entry 2:
    MBR: X=[554000, 1599000], Y=[1462, 1462], Z=[5377, 104059]
    → Node | Leaf: False | Entries: 1
  • Entry 3:
    MBR: X=[182000, 921000], Y=[1196, 1197], Z=[2462, 124716]
    → Node | Leaf: False | Entries: 4

```

B' Φάση: LSH

Τη μέθοδο LSH αποφασίσαμε να την εφαρμόσουμε στο πεδίο “Model name”, αφού είναι το μόνο που οι τιμές του είναι χαρακτήρες και σε τέτοια δεδομένα έχει κυρίως εφαρμογή αυτός ο αλγόριθμος. Αποτελείται από τρία βήματα.

Το πρώτο ονομάζεται “Shingling” και σε αυτό κάθε όνομα μοντέλου μετατρέπεται σε ένα σύνολο από διγράμματα (shingles, $k=2$). Έπειτα φτιάχνεται το λεξιλόγιο μας που αποτελείται από το σύνολο των shingles όλων των ονομάτων των μοντέλων.

Το επόμενο βήμα ονομάζεται “MinHashing” και ξεκινάει με την δημιουργία τυχαίων hash functions. Συγκεκριμένα, η συνάρτηση κατακερματισμού που έχουμε επιλέξει είναι της μορφής $h(x) = (a * x + b) \bmod \text{prime}$. Τα a, b επιλέγονται τυχαία από ένα τεράστιο εύρος τιμών και ο prime είναι ένας πολύ μεγάλος αριθμός ($> 2^{32}$). Σκόπος αυτής της συνάρτησης είναι να προσομοιώσει τις τυχαίες μεταθέσεις που απαιτούνται από τον LSH, αλλά είναι ιδιαίτερα ακριβές σε χώρο και χρόνο. Μία δυάδα (a, b) συνιστούν μία συνάρτηση κατακερματισμού και για κάθε όνομα μοντέλου που έχει μετατραπεί σε `shingle_set` εφαρμόζουμε 20 συναρτήσεις κατακερματισμού. Έτσι, προκύπτει η MinHash υπογραφή του μοντέλου που είναι μία λίστα αποτελούμενη από 20 “τυχαίες” τιμές.

Το τελευταίο βήμα ονομάζεται “Locality Sensitive Hashing”. Εδώ κάθε signature χωρίζεται σε 10 bands με 2 στοιχεία το καθένα. Δημιουργείται μία λίστα (`lsh_buckets`), κάθε στοιχείο της οποίας είναι ένα python dictionary που έχει ως key τη hashed τιμή του band slice (`bucket_key`) και ως value το όνομα του μοντέλου. Η συγκεκριμένη λίστα αποτελείται από 10 στοιχεία, όσα και τα bands και κάθε στοιχείο περιέχει όλα τα dictionaries που αντιστοιχούν στο εκάστοτε band. Αν σε κάποιο band βρεθούν δύο μοντέλα με ίδιο bucket key θεωρούνται παρόμοια.

Για να τρέξουμε τον κώδικα ο χρήστης θα πρέπει να εισαγάγει το όνομα μοντέλου βάσει του οποίου θα τρέξουμε τον LSH για αυτά στο dataset μας. Τότε, θα ακολουθηθεί η ίδια διαδικασία για το query μοντέλο που αναφέρθηκε προηγουμένως για τα μοντέλα του dataset και θα καταλήξουμε σε 10 bucket keys για κάθε band. Για κάθε bucket key που θα προκύπτει θα ελέγχουμε στο αντίστοιχο band του `lsh_buckets`. Αν βρούμε bucket key που να ταυτίζεται θεωρούμε όλα τα μοντέλα που το μοιράζονται ως candidates και τα επιστρέφουμε.

Τέλος, τα candidate shingles και query shingles επαναξιολογούνται με βάση τη Jaccard Similarity τους, ώστε να δοθεί ένα σκορ ομοιότητας και να παρουσιαστούν ως ταξινομημένα αποτελέσματα.

Παρακάτω παραθέτουμε και ένα screenshot από εκτέλεση του κώδικα:

```
Enter the model name to query (e.g., 'Hyundai i10'): 211 Maru Xla

Top-5 models similar to '211 Maru Xla':
2021 Maruti XL6 ZETA MT      → Similarity: 0.286
2021 Maruti XL6 ALPHA AT     → Similarity: 0.273
2021 Maruti XL6 ALPHA MT     → Similarity: 0.273
2011 Maruti Swift VXI        → Similarity: 0.238
2011 Maruti Swift LXI        → Similarity: 0.238
```

Γ' Φάση: Evaluation

Σε αυτό το σημείο καλούμαστε να συγκρίνουμε τις τέσσερις τρισδιάστατες δομές που δημιουργήσαμε και να καταλήξουμε σε ένα συμπέρασμα για την αποδοτικότητα τους. Αυτό αποφασίσαμε να το κάνουμε μετρώντας για τη κάθε δομή το build time, το query time, το χρόνο που χρειάστηκε στον αλγόριθμο LSH να επιστρέψει αποτελέσματα και στον αριθμό των αποτελεσμάτων που επέστρεψε ο LSH.

Για το σκοπό αυτό δημιουργήσαμε το αρχείο κώδικα “evaluation.py”. Εκεί κάνουμε import συναρτήσεις από την κάθε δομή που εκτελούν το χτίσιμο του εκάστοτε δέντρου, αλλά και το ψάξιμο για την ικανοποίηση ενός range query. Το σκεπτικό είναι να τρέξουμε την κάθε δομή υπολογίζοντας τις προαναφερόμενες μετρικές για πληθώρα ερωτημάτων και τέλος να παρουσιάσουμε συγκεντρωτικά τα αποτελέσματα.

Από τη θεωρία γνωρίζουμε τις πολυπλοκότητες των δομών που χτίσαμε και είναι οι παρακάτω:

	Χρόνος	
	Κατασκευή	Range Search
KD tree	$O(n \log n)$	$O\left(n^{1-\frac{1}{3}} + k\right)$
Octree	$O(n \log n)$	$O(n)$
R tree	$O(n \log n)$	$O(n)$
Range Tree	$O(n \log^2(n))$	$O(\log^3(n) + k)$

Και περιμέναμε και αντίστοιχα αποτελέσματα κατά την εκτέλεση του κώδικα μας.

Για την δημιουργία των queries χρησιμοποιήσαμε τη συνάρτηση “**generate_random_query()**”, η οποία σύμφωνα με τις ελάχιστες και μέγιστες τιμές των 3 χαρακτηριστικών του dataset βάσει των οποίων χτίζουμε τις δομές, επιστρέφει ένα range query που αποτελείται από τα τρία ζητούμενα ranges.

Για κάθε δομή έχουμε απο μία συνάρτηση (“**evaluate_range_tree**” για το range tree π.χ.), η οποία θα χτίσει το δέντρο και θα υπολογίσει

το `build_time`, έπειτα θα παράξει μέσω της `“generate_random_query()”` 50 τυχαία queries, θα εκτελέσει το range search για το καθένα, θα υπολογίσει το `query_time` και τέλος θα υπολογίσει και το χρόνο που χρειάζεται να τρέξει ο LSH για τους candidates που επιστρέφει κάθε φορά, μαζί με το πλήθος τους.

Τέλος, εκτυπώνουμε τα αποτελέσματα, ένα παράδειγμα εκ των οποίων απεικονίζεται στο παρακάτω screenshot

=== Final Performance Comparison ===				
Structure	Build Time (s)	Avg Query Time (ms)	Avg LSH Time (ms)	Avg # Candidates
Range Tree + LSH	0.5129	0.11	25.99	32.44
KD-Tree + LSH	0.0067	0.27	19.52	32.36
QuadTree + LSH	0.0105	0.23	30.17	34.56
R-Tree + LSH	0.2999	0.30	39.29	39.18

Παρατηρούμε ότι το Range Tree είναι με εμφανή διαφορά το πιο αργό όσον αφορά το χτίσιμο του δέντρου, κάτι που περιμέναμε λόγω πολυπλοκότητας ($O(n \log^2 n)$), όμως αναπληρώνει σε query time που σημειώνει το χαμηλότερο λόγω της εμφωλευμένης δομής τους. Τα KD-Tree και Quad Tree παρουσιάζουν πολύ χαμηλούς χρόνους κατασκευής, καθιστώντας τα ιδανικά για δυναμικά datasets, ενώ και τα query time τους είναι χαμηλά. Το R-Tree, αν και κατασκευάζεται σε $O(n \log n)$, εδώ εμφάνισε μεγαλύτερους χρόνους τόσο στο χτίσιμο, όσο και σε query time, αλλά και στις μετρικές σχετικές με το LSH. Αυτό αποδίδεται στα πιο ευέλικτα και επικαλυπτόμενα πλαίσια οριοθέτησής του, τα οποία είχαν ως αποτέλεσμα μεγαλύτερο μέσο αριθμό candidates που περνούσαν στη φάση LSH.

Συγκεκριμένα, ο χρόνος εκτέλεσης του LSH βρέθηκε να συσχετίζεται άμεσα με τον αριθμό των candidates που επιστρέφονται από κάθε δομή, επιβεβαιώνοντας ότι το αρχικό φιλτράρισμα των δεδομένων με

βάση το εύρος τιμών και η χωρική διάσπαση που προκαλεί η κάθε δομή επηρεάζει σημαντικά την απόδοση της τελικής αναζήτησης ομοιότητας.

Συμπερασματικά, τα αποτελέσματα επιβεβαιώνουν τα γνωστά από τη θεωρία μας δεδομένα και αναδεικνύουν την αποτελεσματικότητα της κάθε δομής σε συνδυασμό με τη μέθοδο LSH.